Jonas Low - Project Portfolio Page

Overview

Business Management System (BMS) is a command-line app for owners of small, growing businesses. It keeps the product catalogue, stock, customer orders, sales and expenses in one place, and keeps stock in step with every order. It is written in Java and has about 6 kLoC, including tests.

Summary of Contributions

- **Code contributed**: RepoSense link

- **Feature: Adding an order** (`order add`)

  - What it does: records a customer's order for one or more products and deducts the units from stock.

  - Justification: orders are the core of the app. Sales, receipts, payments and order details all read from the orders this command creates.

  - Highlights: the parser reads repeated `p/ q/` pairs and links each quantity to the product before it. A prefix only counts at the start of a word, so product names containing `c/` or `p/` are not split. The command checks every line before changing anything, so an order with one bad line is rejected whole and stock is untouched. Each line stores the unit price at the time of sale, so later price changes do not alter past orders. A product repeated within one order is rejected, because two lines could each pass the stock check while together asking for more than is in stock.

- **Feature: Cancelling and listing orders** (`order cancel`, `order list`)

  - What it does: cancels an order and returns its units to stock; lists every order with its status.

  - Highlights: cancelled orders are kept and marked rather than deleted, so order numbers are never reused. Cancellation is marked before stock is restored, so stock can never be restored twice. Sales and receipts exclude cancelled orders.

- **Feature: Editing an order** (`order edit`)

  - What it does: changes an active order's customer, product or quantity, moving stock to match.

  - Highlights: the most involved of the order features. Every combination of changes (same product with more or fewer units, or a different product) has to move the right units in and out of stock. The command plans the full change and checks stock before applying anything, so a failed edit needs no rollback. For the same product only the extra units must be in stock; an unchanged product keeps its original price. An assertion checks that stock units are only moved, never created or lost.

- **Feature: Discounting an order** (`order discount`)

  - What it does: applies, replaces or removes a percentage discount on an active order.

  - Highlights: only the percentage is stored, and the total is recalculated from the order's lines when needed, so an edited order keeps its discount with no extra code. Discount amounts are calculated with `BigDecimal` and rounded half-up to the cent, because a `double` stores 12.5% of $9.40 as 1.17499... and would round it down.

- **Testing**: wrote 34 JUnit tests for the order commands (`AddOrderCommandTest`, `CancelOrderCommandTest`, `EditOrderCommandTest`, `DiscountOrderCommandTest`), covering stock movement, rejected input leaving state unchanged, and rounding edge cases. Added assertions to `AddOrderCommand`.

- **Documentation**:

  - User Guide: wrote section 3 (Orders), covering adding, cancelling, listing, editing and discounting orders, with example outputs for each; fixed formatting in the Command Summary.

  - Developer Guide: wrote the Orders section of Design & Implementation, covering adding, cancelling, listing, editing and discounting orders, with design considerations for each.

- **Contributions to team-based tasks**:

  - Fixed the CI build, which was failing on every PR: restored Javadoc openers in `Ui.java` lost in a merge, added the missing order commands to `help order`, and updated the outdated help tests (#73).

  - Updated the About Us page (#31).

- **Review/mentoring contributions**:

  - Reviewed and merged PRs #70 (stock low command; resolved its merge conflict with master) and #71 (DG user stories).

- **Contributions beyond the project team**:

  - TODO: add bugs you report in other teams' products during the PE dry run, and any forum posts.